Spring Boot Escrow Marketplace System

1. System Objective

```
Design a financial-grade escrow system for a farm equipment rental marketplace
with proper double-entry accounting, idempotency handling, immutable ledgers,
and strict state validation.
```

2. Core Financial Invariants

```
- Every financial movement must be recorded as a transaction.
- Each transaction must have at least two ledger entries.
- Total debit must equal total credit.
- Ledger entries are immutable.
- Account balance = Sum of ledger entries.
```

3. Architecture Layers

```
Controller Layer – Handles REST APIs
Service Layer – Business + transactional logic
Repository Layer – JPA/Hibernate persistence
Database – PostgreSQL
```

4. State Models

```
Booking Status:
CREATED → CONFIRMED → PAID_LOCKED → COMPLETED → RELEASED → CANCELLED

Payment Status:
INITIATED → ORDER_CREATED → CAPTURED → FAILED → REFUNDED

Financial Transaction Status:
PENDING → POSTED → REVERSED → FAILED
```

5. Database Schema Overview

```
accounts:
- id (UUID)
- owner_id
- account_type
- currency
- created_at

transactions:
- id (UUID)
- reference_id
```

```
- reference_type
- status
- idempotency_key
- created_at

ledger_entries:
- id (UUID)
- transaction_id (FK)
- account_id (FK)
- entry_type (DEBIT/CREDIT)
- amount (NUMERIC 18,2)
- created_at
(Immutable – No update/delete)

idempotency_keys:
- key (unique)
- request_hash
- response_snapshot
- created_at

audit_logs:
- id
- actor
- action
- reference_id
- metadata (jsonb)
- created_at
```

6. Escrow Flow

```
1. Booking created.
2. Razorpay order created.
3. Webhook verifies HMAC signature.
4. On success:
   DEBIT RENTER_CLEARING
   CREDIT ESCROW
5. Booking updated to PAID_LOCKED.
```

7. Settlement Flow

```
Scheduled job detects completed rentals.

Transaction:
DEBIT ESCROW
CREDIT LENDER_WALLET
CREDIT PLATFORM_REVENUE

Booking updated to RELEASED.
Unique constraint prevents double settlement.
```

8. Withdrawal Flow

```
1. Lock lender wallet (SELECT FOR UPDATE or optimistic locking).
2. Compute balance from ledger.
3. Prevent negative balance.
4. Create transaction:
   DEBIT LENDER_WALLET
   CREDIT PLATFORM_BANK
```

9. Concurrency & Safety

- Use @Transactional for all money movement.
- Enforce debit = credit before posting.
- Use idempotency keys for webhook & withdrawals.
- Prevent invalid booking state transitions.
- Never store mutable wallet balance.

10. Advanced Enhancements

- Reversal transactions (never delete).
- Razorpay reconciliation table.
- Outbox pattern for event publishing.
- Integration tests with race conditions.
- Database constraints for invariants.